DOCUMENT D'ARCHITECTURE TECHNIQUE (DAT)

DEVOPS ASSISTANT BOT - AFRICA TECH TOUR 2025

PRÉSENTÉ PAR :

YEBADOKPO Sidoine
Étudiant Boursier - Programme Africa Tech Tour 2025
Contact : Bénin, +2290196911346/ sidoineko@gmail.com
Portfolio : huggingface.co/spaces/Sidoineko/portfolio

SOMMAIRE

1. **PRÉSENTATION DU PROJET**
2. **ARCHITECTURE FONCTIONNELLE**
3. **ARCHITECTURE TECHNIQUE DÉTAILLÉE**
4. **LOGIQUE MÉTIER ET INTELLIGENCE ARTIFICIELLE**
5. **GESTION DES DONNÉES ET PERSISTANCE**
6. **SÉCURITÉ ET AUTHENTIFICATION**
7. **PIPELINE CI/CD ET AUTOMATISATION**

8. **CONTENEURISATION ET INFRASTRUCTURE**
9. **RÉSILIENCE ET HAUTE DISPONIBILITÉ**
10. **MONITORING ET OBSERVABILITÉ**
11. **GUIDE D'INSTALLATION ET EXPLOITATION**
12. **BILAN ET PERSPECTIVES**

13. **ANNEXES ET RÉFÉRENCES**

---

1. PRÉSENTATION DU PROJET

1.1 Contexte et Vision

Le **DevOps Assistant Bot** est né d'un besoin concret identifié lors du parcours DevOps de l'**Africa Tech Tour 2025**. Les apprenants font souvent face à une montagne de concepts complexes (Docker, Kubernetes, CI/CD, Terraform) et à des erreurs de déploiement parfois cryptiques.

Ce projet propose un compagnon intelligent capable d'expliquer, de guider et de dépanner les juniors en s'appuyant sur le corpus pédagogique officiel de la formation.

1.2 Ecran d'acceuil/Register/Login

Figure 1 : Ecran d'acceuil DevOps Assistant Bot

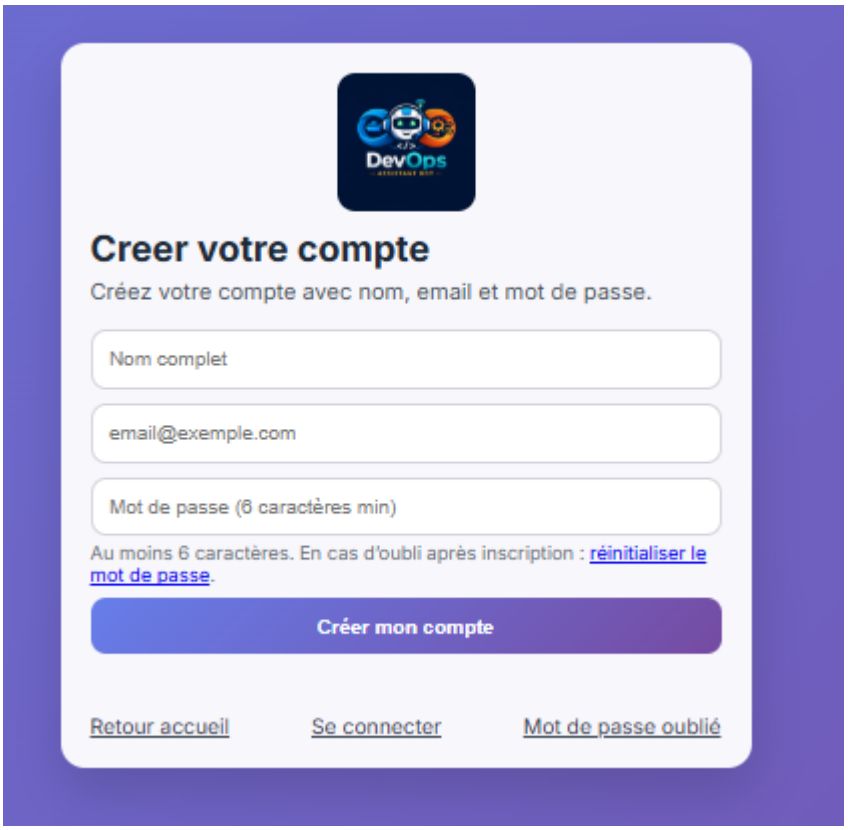


Figure 2 : Ecran Login/Register DevOps Assistant Bot

1.3 Objectifs Stratégiques

- **Accessibilité** : Fournir une interface intuitive en français pour poser des

questions techniques.

- **Fiabilité** : Garantir des réponses fondées sur des sources vérifiées (RAG).
- **Souveraineté** : Permettre un fonctionnement “Offline” sans dépendance aux API Cloud coûteuses.

- **Industrialisation** : Démontrer une chaîne CI/CD complète et une infrastructure conteneurisée.

---

2. ARCHITECTURE FONCTIONNELLE

L'application est conçue selon un modèle **Client-Serveur** moderne utilisant des **Websockets** pour une interactivité en temps réel.

2.1 Interface Utilisateur (Frontend)

Le frontend est une application "Vanilla JavaScript" optimisée pour la performance et la légèreté.

- **Tableau de bord de chat** : Interface fluide pour la conversation.
- **Gestionnaire de documents** : Upload de fichiers PDF/LOG pour enrichir le contexte.
- **Console de configuration** : Choix dynamique du moteur d'IA (Gemini, OpenAI, Local).

2.2 Rendu du Bot en action

Le bot fournit des réponses structurées, citant les sources issues des cours de l'Africa Tech Tour.

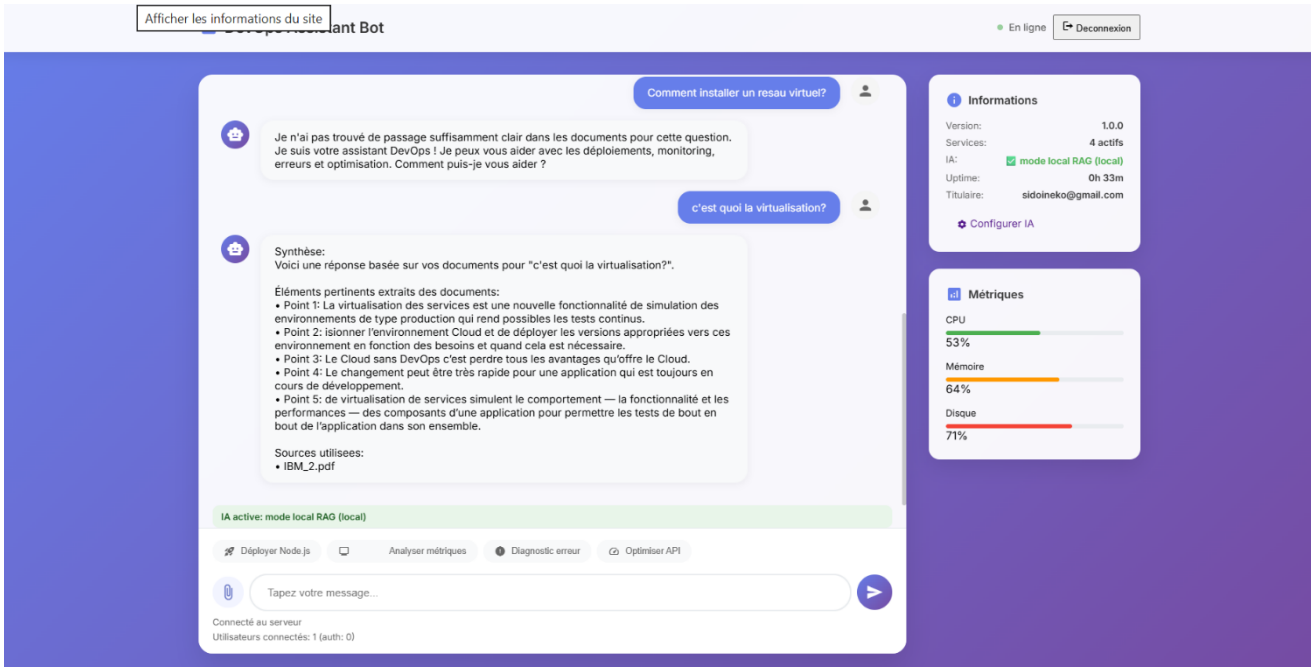


Figure 3 : Rendu du bot suite à une question technique DevOps (Mode RAG-Offline)

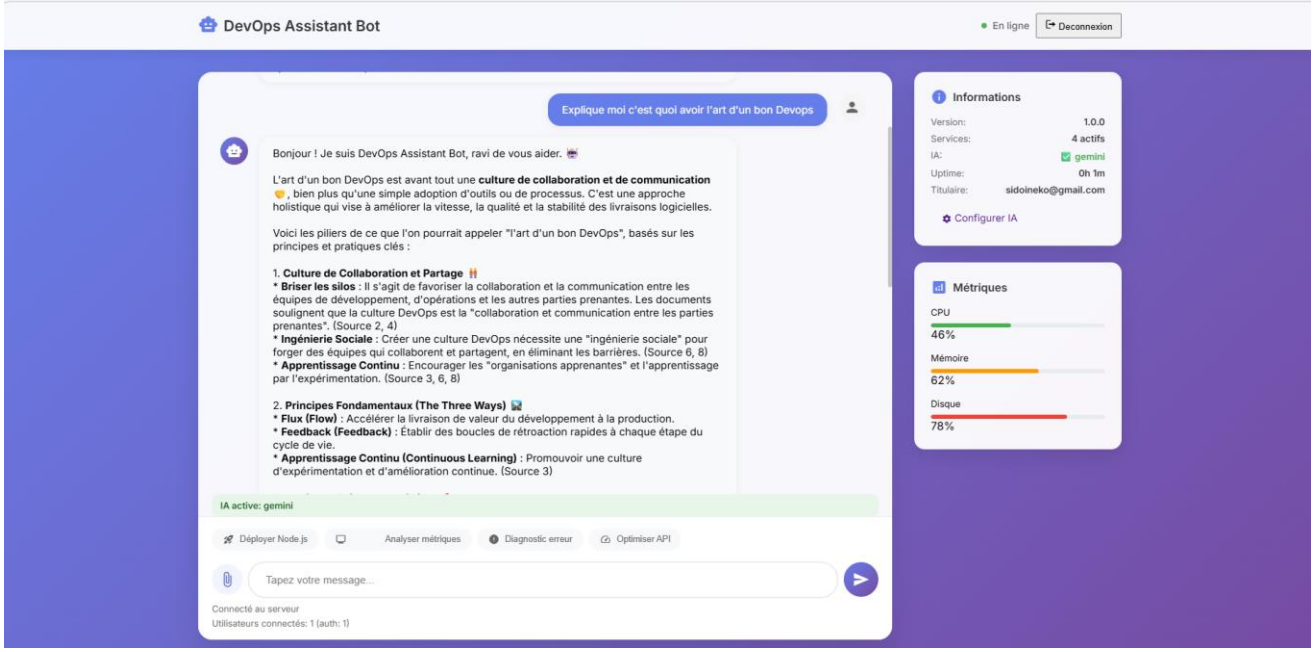
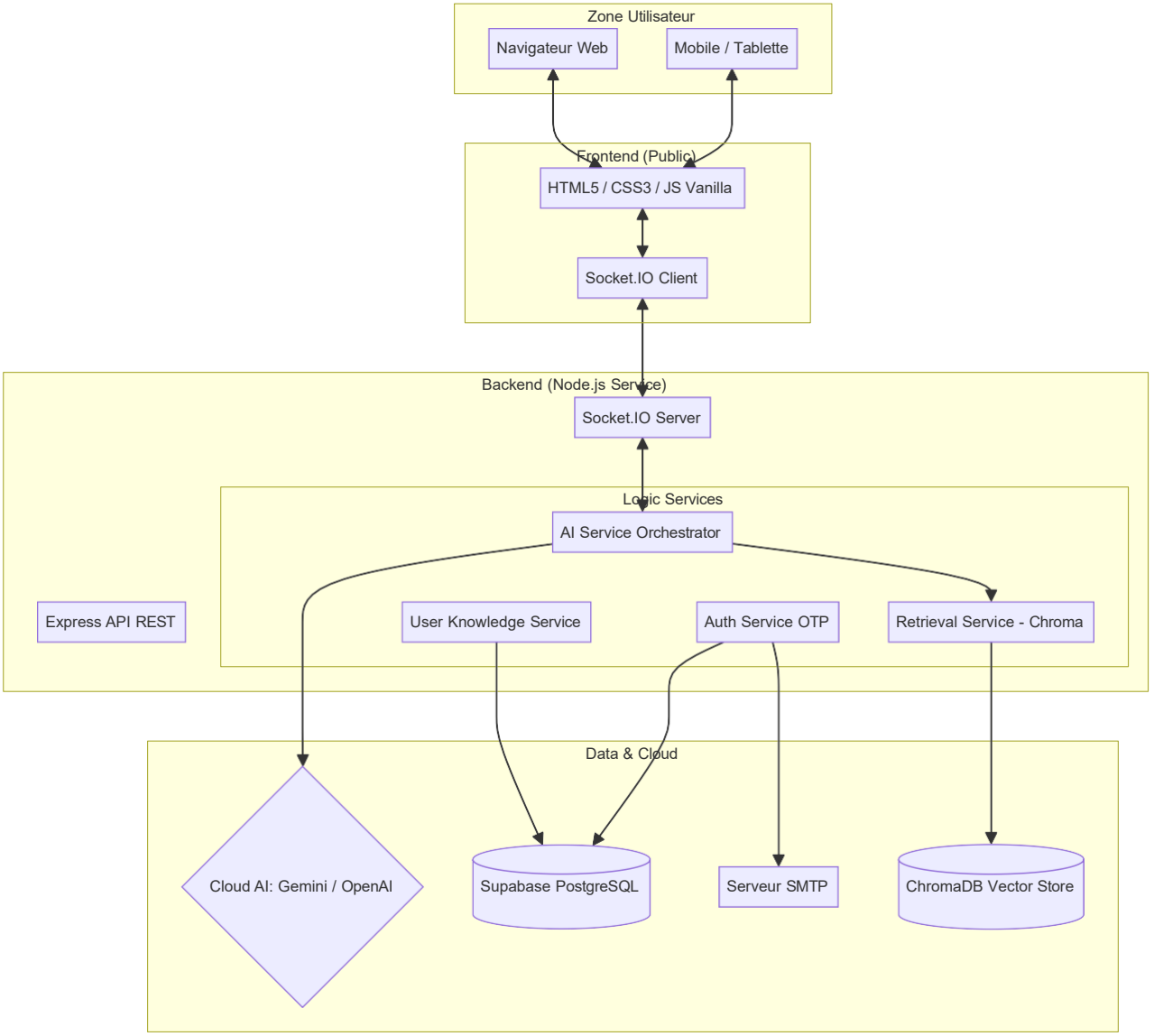


Figure 4 : Rendu du bot suite à une question technique DevOps (Mode Online avec l'IA de Gemini)

3. ARCHITECTURE TECHNIQUE DÉTAILLÉE

3.1 Schéma d'Architecture Global



4. LOGIQUE MÉTIER ET INTELLIGENCE ARTIFICIELLE

4.1 Le moteur RAG (Retrieval Augmented Generation)

Contrairement à un chatbot classique, notre solution ne "devine" pas. Elle **cherche** d'abord l'information dans les cours de l'Africa Tech Tour stockés dans **ChromaDB**.

1. **Ingestion** : Les PDF de cours sont découpés en segments (chunks) et transformés en vecteurs mathématiques.
2. **Récupération** : À chaque question, le système cherche les segments les plus proches sémantiquement.
3. **Génération** : L'IA reçoit la question + les segments de cours pour produire une réponse sourcée.

4.2 Multimodalité et Fallback

Le système est capable d'analyser des captures d'écran d'erreurs (via Gemini Vision) et de basculer sur un mode **Local-RAG** si la connexion internet est coupée ou si les quotas d'API sont atteints.

---

5. GESTION DES DONNÉES ET PERSISTANCE

5.1 Supabase (PostgreSQL)

Nous utilisons Supabase pour sa robustesse et ses fonctionnalités de sécurité natives.

- **Table** `users` : Profils et métadonnées.
- **Table** `user_knowledge_chunks` : Stockage des documents personnels uploadés par les apprenants.
- **Table** `conversations` : Historique complet pour la continuité pédagogique.

5.2 ChromaDB (Vector Database)

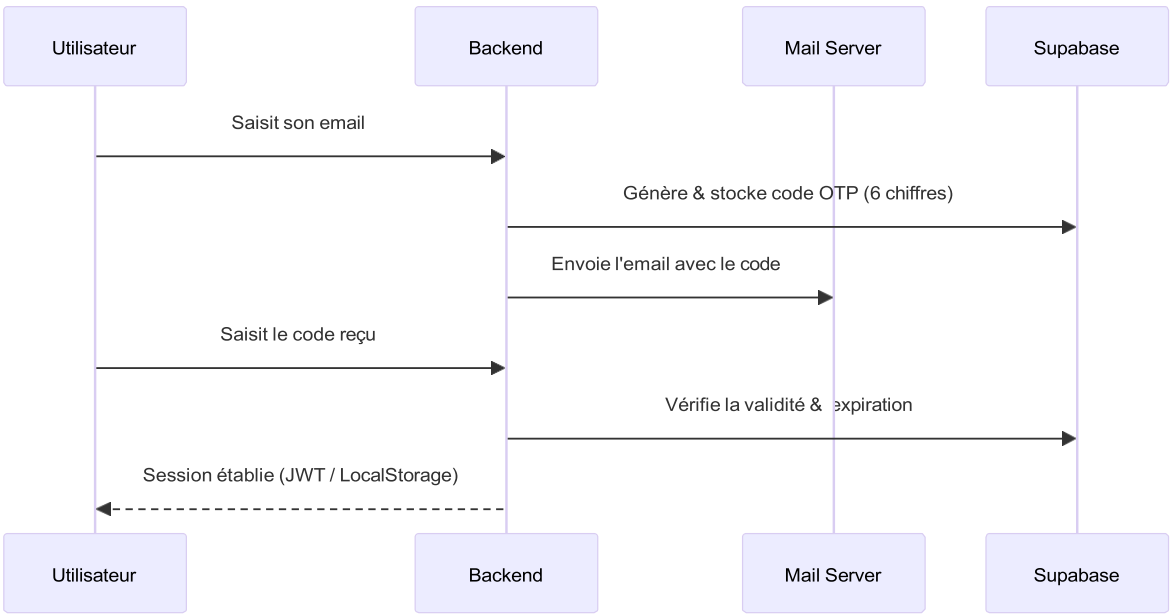
Base de données spécialisée pour le stockage des "embeddings" (représentations vectorielles du texte). Elle permet une recherche de similarité ultra-rapide, indispensable pour le RAG.

---

6. SÉCURITÉ ET AUTHENTIFICATION

6.1 Authentification par OTP (One-Time Password)

Pour simplifier l'expérience utilisateur tout en garantissant la sécurité, nous avons implémenté un système de connexion par code unique envoyé par email.



6.2 Isolation des données (RLS)

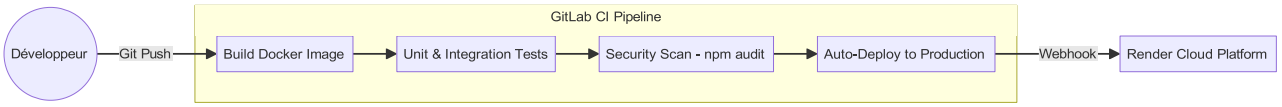
Grâce au **Row Level Security (RLS)** de PostgreSQL, chaque apprenant ne peut accéder qu'à ses propres documents et conversations. Même en cas de faille applicative, les données restent cloisonnées au niveau de la base de données.

---

7. PIPELINE CI/CD ET AUTOMATISATION

Le projet incarne la culture DevOps par une automatisation totale de la chaîne de livraison via **GitLab CI**.

7.1 Cycle de vie du code (Workflow CI/CD)



7.2 Détail des étapes

1. **Build** : Construction de l'image Docker multi-plateforme et stockage dans le registre privé GitLab.
2. **Test** : Exécution des tests automatisés (Jest) avec des services éphémères (PostgreSQL, Redis) pour garantir la qualité.

3. **Security** : Analyse des dépendances pour détecter d'éventuelles vulnérabilités critiques.
4. **Deploy** : Mise à jour automatique de l'environnement de production sur Render.

---

8. CONTENEURISATION ET INFRASTRUCTURE

8.1 Docker au cœur de la solution

L'application est packagée dans une image Docker légère basée sur `node:18-alpine`.

- **Dockerfile** : Utilisation du multi-stage build pour réduire la taille de l'image finale (sécurité et rapidité).
- **Docker Compose** : Orchestration locale permettant de lancer l'application et sa base vectorielle ChromaDB en une seule commande.

8.2 Hébergement Cloud (Render)

Le choix de **Render** permet une gestion "Infrastructure as Code" via le fichier `render.yaml`, assurant la reproductibilité de l'environnement de production.

---

9. RÉSILIENCE ET HAUTE DISPONIBILITÉ

Le bot est conçu pour ne jamais "tomber" complètement :

- **Healthchecks** : Route `/health` surveillée par Render pour redémarrer le conteneur en cas d'échec.
- **Chroma Fallback** : Capacité de basculer entre plusieurs serveurs de base de données vectorielle.
- **Backup Manager** : Système de restauration automatique des index vectoriels à partir de sauvegardes ZIP.

---

10. MONITORING ET OBSERVABILITÉ

Pour assurer la qualité de service, nous suivons plusieurs métriques clés :

- **Métrique d'usage** : Nombre d'utilisateurs connectés en temps réel (via Socket.IO).
- **Logs d'erreurs** : Centralisés dans Supabase pour un diagnostic rapide.
- **Performance IA** : Temps de réponse des différents providers (Gemini vs OpenAI).

---

11. GUIDE D'INSTALLATION ET EXPLOITATION

11.1 Pré-requis

- Docker & Docker Compose
- Compte Supabase (URL + Clé API)
- Clé API Gemini ou OpenAI (Optionnel pour le mode local)

11.2 Lancement rapide

```
git clone https://github.com/Sidoine1991/DevOps-Assistant-Bot.git
cd DevOps-Assistant-Bot
docker-compose up -d
```

---

12. BILAN ET PERSPECTIVES

Ce projet démontre qu'une architecture simple mais robuste peut répondre à des besoins pédagogiques complexes.

Évolutions prévues :

- Support des fichiers Terraform et Ansible pour analyse de conformité.
- Intégration d'un module de "Live Coding" assisté.
- Déploiement sur un cluster Kubernetes (K8s) pour tester la scalabilité horizontale.

---

13. ANNEXES ET RÉFÉRENCES

- **URL Production** : https://devops-assistant-bot-1wcp.onrender.com/

- **Dépôt GitHub** : Sidoine1991/DevOps-Assistant-Bot
- **Portfolio Sidoine YEBADOKPO** : huggingface.co/spaces/Sidoineko/portfolio

---

Document produit par YEBADOKPO Sidoine dans le cadre de la certification Africa Tech Tour 2025.